

Functions and Procedures in SQL

Detailed Notes with Examples

Functions in SQL

Definition

A **Function** is a named PL/SQL or SQL block that:

- Performs a specific task or calculation.
- Returns exactly one value (using **RETURN** statement).
- Can be used in SQL statements wherever expressions are allowed.

Characteristics of Functions

- Must return a value.
- Can accept parameters (IN mode).
- Can be used in **SELECT**, **WHERE**, **ORDER BY** clauses.
- Cannot perform transaction control (e.g., **COMMIT** or **ROLLBACK**).

Syntax: Creating a Function

```
1 CREATE OR REPLACE FUNCTION function_name (parameter_list)
2 RETURN return_datatype
3 IS
4 BEGIN
5     -- Function logic
6     RETURN some_value;
7 END;
```

Example 1: Function to Calculate Area of Circle

```
1 CREATE OR REPLACE FUNCTION circle_area(radius NUMBER)
2 RETURN NUMBER
3 IS
4     area NUMBER;
5 BEGIN
6     area := 3.1416 * radius * radius;
7     RETURN area;
8 END;
```

Usage Example

```
1 SELECT circle_area(10) AS area FROM dual;
```

Procedures in SQL

Definition

A **Procedure** is a named PL/SQL or SQL block that:

- Performs a task or series of operations.
- Does not return a value directly but can return multiple values via OUT parameters.
- Used to encapsulate complex business logic or DML operations.

Characteristics of Procedures

- May accept parameters: IN, OUT, or IN OUT modes.
- Can perform transaction control (COMMIT, ROLLBACK).
- Cannot be used directly in SQL statements.
- Can be called from other PL/SQL blocks or applications.

Syntax: Creating a Procedure

```
1 CREATE OR REPLACE PROCEDURE procedure_name (parameter_list)
2 IS
3 BEGIN
4     -- Procedure logic
5 END;
```

Example 2: Procedure to Update Employee Salary

```
1 CREATE OR REPLACE PROCEDURE update_salary (  
2     emp_id IN NUMBER,  
3     new_salary IN NUMBER  
4 )  
5 IS  
6 BEGIN  
7     UPDATE employees  
8     SET salary = new_salary  
9     WHERE employee_id = emp_id;  
10  
11     COMMIT;  
12 END;
```

Calling the Procedure

```
1 BEGIN  
2     update_salary(101, 9000);  
3 END;
```

Parameters Modes

Parameter Modes

- **IN** – Passes a value into the procedure/function. Default mode.
- **OUT** – Returns a value to the caller.
- **IN OUT** – Passes a value in and returns a value back.

Example 3: Procedure with IN OUT and IN OUT

```
1 CREATE OR REPLACE PROCEDURE process_numbers (  
2     num1 IN NUMBER,  
3     num2 IN OUT NUMBER,  
4     result OUT NUMBER  
5 )  
6 IS  
7 BEGIN  
8     num2 := num2 + 10;  
9     result := num1 + num2;  
10 END;
```

Differences Between Functions and Procedures

Comparison Summary

Function	Procedure
Must return a single value using RETURN	Does not return value via RETURN (uses OUT params)
Can be called in SQL statements	Cannot be called directly in SQL statements
Primarily used for computations and returning values	Primarily used for performing actions or transactions
Cannot perform COMMIT or ROLLBACK	Can perform transaction control
Usually has only IN parameters	Can have IN, OUT, and IN OUT parameters

Additional Examples

Example 4: Function to Check Even/Odd

```
1 CREATE OR REPLACE FUNCTION is_even(n NUMBER)
2 RETURN VARCHAR2
3 IS
4 BEGIN
5     IF MOD(n, 2) = 0 THEN
6         RETURN 'EVEN';
7     ELSE
8         RETURN 'ODD';
9     END IF;
10 END;
```

Example 5: Procedure to Insert a New Employee

```
1 CREATE OR REPLACE PROCEDURE add_employee (  
2     emp_id IN NUMBER,  
3     emp_name IN VARCHAR2,  
4     emp_dept IN VARCHAR2  
5 )  
6 IS  
7 BEGIN  
8     INSERT INTO employees(employee_id, name, department)  
9     VALUES (emp_id, emp_name, emp_dept);  
10  
11     COMMIT;  
12 END;
```

Summary

Key Takeaways

- **Functions** return a single value and can be used in SQL queries.
- **Procedures** perform operations, may return multiple values via parameters, but cannot be used inside SQL statements.
- Parameters can have modes: IN, OUT, and IN OUT.
- Both improve modularity, code reuse, and maintainability of database programs.